Competitive Exam Practice 8 — Answer Key

Searching and the Elementary Sorts

Auqib Hamid Lone

Department of Computer Science & Engineering, GCET Ganderbal

October 15, 2026

Concepts That Click

Q1 (D) $(n+1)/2$**.** A successful search is equally likely to find the key in any of the n positions; reaching position i costs i comparisons, so the average is $\frac{1}{n}(1+2+\cdots+n) = \frac{n(n+1)/2}{n} = \frac{n+1}{2}$. The local book's answer key lists this as option C under its own option ordering; the value $(n+1)/2$ is the authenticated answer (GATEOverflow; GfG options).

Q2 (C) $Y = [2\ 2\ 2\ 2\ 2\ 2\ 2\ 2\ 2\ 2]$ and $x > 2$. With every $Y[k] = 2 < x$, the branch `i = k` keeps `i` fixed at the same `k` while `j` stays 9; once `i` reaches 8, `k` computes to 8 again, so `(Y[k]!=x) && (i<j)` stays true and the loop never terminates — an infinite loop (the program "fails"). For (A), (B), (D) the search either hits x or drains the range and prints a correct answer; e.g. (D) with $x = 8$ reaches `Y[3] = 8` directly. (Local book key 1.37.4 = C; Electrical4u/GfG agree.)

Q3 (C) Selection sort. Selection sort makes exactly $n-1$ swaps in total (one per outer iteration), the minimum of any of the four in-place sorts; quick, insertion, and heap sort can each perform $\Theta(n^2)$ moves/swaps in the worst case. (Local book key 1.40.12 = C; GfG "DSA Set 7" answer (C).)

Q4 (C) Insertion sort. The array $[23, 32, 45, 69, 72, 73, 89, 97]$ is already sorted in ascending order, which is insertion sort's best case: it makes one comparison per element for a total of $n-1 = 7$ comparisons and no shifts. Mergesort is $\Theta(n \log n)$ (≈ 24 comparisons); selection sort is always $n(n-1)/2 = 28$; quicksort with the last element as pivot degrades to $\Theta(n^2)$ on sorted input. So insertion sort uses the fewest. (Local book key 1.40.16 = C.)

Q5 (B) It maintains the relative order of occurrence of non-distinct elements. A sort is **stable** exactly when equal keys retain their original relative order; asymptotic time, use of divide-and-conquer, and space usage are separate properties and do not define stability. (Local book key 1.40.8 = B; GfG options verbatim.)

Q6 9**.** The predicate $P(k) = (k^2 \le 90)$ is true up to $k = 9$ ($9^2 = 81 \le 90$) and false from $k = 10$ ($10^2 = 100 > 90$). Binary search on the answer space 0..90 returns the largest feasible k, i.e. $\lfloor\sqrt{90}\rfloor = 9$. (One probe: `mid`= 45 false; 22 false; 10 false; 4 true; 7 true; 8 true; 9 true $\rightarrow$ answer 9.) Verified.

Q7 (A) "bo" (length 2). In sorted order the set is "book" < "boot" < "bough". The LCP of the whole set equals the LCP of the first and last strings, "book" and "bough": they agree on 'b' and 'o' and first disagree at position 2 ('o' vs 'u'), so the LCP is "bo", length 2. Verified.

Q8 **(A)** $[1,\ 4,\ 2,\ 5,\ 8]$ **with 2 swaps.** One left-to-right pass compares the adjacent pairs $(4,1)$, $(4,5)$, $(5,2)$, $(5,8)$: $(4,1)$ swaps $\to [1,4,5,2,8]$; $(4,5)$ is in order; $(5,2)$ swaps $\to [1,4,2,5,8]$; $(5,8)$ is in order. Result $[1,4,2,5,8]$ after exactly 2 swaps. Verified.

Q9 **(B).** Counting sort is *not* a comparison sort; it tallies keys, forms prefix sums, and scatters in $\Theta(n+k)$ time, stable thanks to the reverse scatter, but it needs a $\Theta(k)$ tally plus a $\Theta(n)$ output buffer, so it is stable yet not in-place. (A) contradicts both its non-comparison nature and its linear time; (C) is false on both counts; (D) is false — a huge key range blows up the tally array. Verified against the deck's Bill table.

Q10 **5.** On an already-ascending array the inner loop test `A[j] > key` is evaluated once per outer iteration and is immediately false (so j drops below 0 and the loop stops). The outer loop runs for $i = 1, 2, 3, 4, 5$, giving $n - 1 = 5$ comparisons in total — insertion sort's adaptive best case. Verified.

Concepts That Click